

Computer Vision Lab 03 - Code + Exam Cheat Sheet

How to use this sheet in the coding exam

For every code problem, identify these four things first:

- **INPUT:** image/points and their coordinate format.
- **OPERATION:** the transformation or OpenCV function that performs it.
- **OUTPUT:** transformed image, matrix, aligned view, panorama, etc.
- **PARAMETERS:** angle, scale, shear, translation, point correspondences, output size.

Important source note: The 20-page Lab 03 Manual is primarily theoretical. It contains equations, diagrams, matrix forms, and references to OpenCV transformation functions, but no standalone Python/OpenCV code block to copy. Therefore the code below is **task-solution code/templates**, derived from the manual's mathematics and the Lab 03 task sheet, not copied from a nonexistent manual code block.

PART A - LAB 03 MANUAL: PAGE-BY-PAGE CODE AUDIT

Page 1

Topic: Lab 03 - Geometric Transformation cover page.

Code present? No.

What to remember: The lab is about changing spatial position/orientation/scale/shape of image content.

Page 2

Topic: Table of contents.

Code present? No.

Topic map:

```

2D Geometric Transformations
|
+-- Affine
|   |
|   +-- Linear
|   +-- Translation
|   +-- Scale
|   +-- Rotation
|   +-- Shear
|   +-- Similarity
|   +-- Rigid
|
+-- Projective (Homography)
    |
    +-- Perspective
  
```

Page 3

Topic: Objectives + definition of geometric transformation.

Code present? No standalone code.

Core idea:

```

Geometric transformation -> changes WHERE image information is located.
Photometric transformation -> changes pixel appearance/intensity.
  
```

Typical geometric notation:

```

(x, y) -> (x', y')
  
```

Page 4

Topic: Geometric vs photometric transformations.

Code present? No standalone code.

Geometric examples: translation, rotation, scaling, shearing, perspective.

Photometric examples: operations that change pixel values rather than pixel locations.

MCQ memory:

```
GEOMETRIC = location/geometry
PHOTOMETRIC = intensity/appearance
```

Page 5

Topic: Transformation hierarchy + start of image coordinate system.

Code present? No.

Hierarchy to recognize:

```
2D geometric
|
+-- Affine
|   +-- Linear
|   +-- Translation
|   +-- Rotation
|   +-- Scale
|   +-- Shear
|   +-- Similarity
|   +-- Rigid
|
+-- Projective / Homography
    +-- Perspective
```

Page 6

Topic: Image coordinate system.

Code present? No code block, but important OpenCV convention.

```
(0,0) -----> x
|
|
|
v
y
```

- origin = top-left
- x increases to the right
- y increases downward

Example in the manual:

```
width  = 640
height = 480
```

Page 7

Topic: Coordinate indexing + 2D transformation.

Code present? No standalone code.

Important distinction:

```
image[y, x]
```

versus a geometric point:

```
(x, y)
```

2D transformation:

$$(x, y) \rightarrow (x', y')$$

The manual gives the typical 2D homogeneous/affine representation as 2×3 for OpenCV affine warping.

Page 8

Topic: OpenCV transformation matrix sizes + 3D transformation.

Code present? No full code block, but two OpenCV function names are explicitly referenced:

```
cv2.warpAffine()  
cv2.warpPerspective()
```

Manual takeaway:

```
warpAffine      -> accepts a 2 x 3 matrix  
warpPerspective -> uses a 3 x 3 matrix
```

3D concept:

$$(x, y, z) \rightarrow (x', y', z')$$

Typical homogeneous 3D representation:

4 x 4 matrix

Page 9

Topic: Transformation matrix.

Code present? No.

2D linear matrix:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

and:

$$p' = A p$$

The matrix maps the original coordinate vector to a new coordinate vector.

Page 10

Topic: Reading the transformation matrix / basis vectors.

Code present? No.

The columns of:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

represent the transformed basis vectors.

1st column = transformed x-basis
2nd column = transformed y-basis

General intuition:

a, d -> diagonal effects such as axis scaling
b, c -> mixing/shearing/rotation effects

Page 11

Topic: Affine transformation introduction.

Code present? No.

Core affine equation:

$$p' = A p + b$$

where:

A = linear transformation matrix

p = original coordinate vector

b = translation vector

Page 12

Topic: Affine transformation properties.

Code present? No.

Affine transformation preserves:

- collinearity
- parallelism
- ratios along a line

A grid can become a tilted parallelogram-like grid, but parallel lines remain parallel.

Page 13

Topic: Linear transformation.

Code present? No.

Linear transformation:

$$p' = A p$$

It is affine without a translation vector.

Page 14

Topic: Expanded linear equations + scaling, rotation, reflection.

Code present? No standalone Python code.

Expanded form:

$$\begin{aligned}x' &= a x + b y \\ y' &= c x + d y\end{aligned}$$

Scaling

$$\begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

Rotation

$$\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

Reflection across X-axis

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Page 15

Topic: Shearing + invariant origin.

Code present? No.

Horizontal shear:

$$\begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$$

which gives:

$$\begin{aligned}x' &= x + k y \\y' &= y\end{aligned}$$

Linear transformations keep the origin fixed.

Page 16

Topic: Limitation of the anchored origin.

Code present? No.

Critical rule:

$$A \theta = \theta$$

Therefore a pure 2 x 2 linear matrix cannot translate an object.

It can:

- rotate
- scale
- shear
- reflect

but cannot slide the entire coordinate system.

Page 17

Topic: Homogeneous coordinates.

Code present? No standalone Python code.

Convert:

$$(x, y) \rightarrow (x, y, 1)$$

Affine transformation can then be represented using a 3 x 3 matrix:

$$\begin{bmatrix} a & b & tx \\ c & d & ty \\ 0 & 0 & 1 \end{bmatrix}$$

and:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & tx \\ c & d & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Page 18

Topic: Composition + projective transformation.

Code present? No full code.

Composition idea:

T, R, S

can be multiplied together into one master matrix.

Projective transformation / homography is more general than affine transformation and allows perspective effects.

Page 19

Topic: Projective mathematical core + denominator effect.

Code present? No.

Homography:

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

For:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

normalize:

$$\begin{aligned} x_{\text{screen}} &= x' / w' \\ y_{\text{screen}} &= y' / w' \end{aligned}$$

The denominator is what gives projective transformation its perspective effect.

Page 20

Topic: Real-world perspective + planar alignment.

Code present? No.

Projective transformations model:

- converging lines
- camera viewpoint changes
- planar alignment
- document perspective correction

Classic example:

angled trapezoid -> flat rectangle

PART B - MASTER LAB 03 FORMULA REFERENCE

1. Linear transformation

$$p' = A p$$

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

Expanded:

$$x' = a x + b y$$

$$y' = c x + d y$$

2. Scaling

$$S = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

If uniform scaling:

$$s_x = s_y = s$$

3. Rotation

$$R(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

Important:

$$\begin{aligned}\cos(-\theta) &= \cos(\theta) \\ \sin(-\theta) &= -\sin(\theta)\end{aligned}$$

To undo a $+\theta$ rotation, use $-\theta$.

4. Horizontal shear

$$\text{Hshear} = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$$

Giving:

$$\begin{aligned}x' &= x + k y \\ y' &= y\end{aligned}$$

To undo a known horizontal shear k :

$$\text{H_inverse} = \begin{bmatrix} 1 & -k \\ 0 & 1 \end{bmatrix}$$

5. Reflection across X-axis

$$\text{Rfx} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

6. Affine transformation

$$p' = A p + b$$

Homogeneous 3 x 3 form:

```
[ a  b  tx
  c  d  ty
  0  0  1 ]
```

7. Translation

```
T =
[ 1  0  tx
  0  1  ty
  0  0  1 ]
```

8. Rigid transformation

Rigid = rotation + translation.

If rotation happens first:

```
H_rigid = T @ R
```

9. Similarity transformation

Similarity = uniform scale + rotation + translation.

If scale -> rotate -> translate:

```
H_similarity = T @ R @ S
```

10. General affine from three point pairs

Use:

$$\begin{aligned}x' &= a x + b y + tx \\ y' &= c x + d y + ty\end{aligned}$$

Unknowns:

a, b, tx, c, d, ty

Three point correspondences give six equations.

System:

$$\begin{aligned}[x_1 \ y_1 \ 1 \ 0 \ 0 \ 0] [a \] &= [x_1'] \\ [0 \ 0 \ 0 \ x_1 \ y_1 \ 1] [b \] &= [y_1'] \\ [x_2 \ y_2 \ 1 \ 0 \ 0 \ 0] [tx] &= [x_2'] \\ [0 \ 0 \ 0 \ x_2 \ y_2 \ 1] [c \] &= [y_2'] \\ [x_3 \ y_3 \ 1 \ 0 \ 0 \ 0] [d \] &= [x_3'] \\ [0 \ 0 \ 0 \ x_3 \ y_3 \ 1] [ty] &= [y_3']\end{aligned}$$

In code, use `np.linalg.solve(A, B)`.

11. Projective / homography

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

Then:

$$\begin{aligned}x' &= (h_{11}x + h_{12}y + h_{13}) / \\ &\quad (h_{31}x + h_{32}y + h_{33}) \\ y' &= (h_{21}x + h_{22}y + h_{23}) / \\ &\quad (h_{31}x + h_{32}y + h_{33})\end{aligned}$$

PART C - TASK 1: THE TINY FINGERPRINT

What the task asks

- Build a 2 x 2 scaling matrix.
- Enlarge the fingerprint by 300% in x and y.
- Keep it centered on the screen.

The task gives:

```
sx = 3  
sy = 3
```

Approach

1. Build the 2 x 2 scale matrix.
2. Scaling around the origin would move the center, so calculate a translation that keeps the center fixed.
3. OpenCV's affine warp uses a 2 x 3 matrix.

Core matrix

```
S = [ 3  0  
      0  3 ]
```

To keep the image center fixed:

```
cx = width / 2  
cy = height / 2  
  
tx = cx - sx*cx  
ty = cy - sy*cy
```

which for $sx=sy=3$ becomes:

```
tx = -2*cx  
ty = -2*cy
```

Complete solution

```
import cv2  
import numpy as np  
  
# Read the fingerprint image  
img = cv2.imread("fingerprint.jpg")  
  
if img is None:  
    raise FileNotFoundError("Could not find fingerprint.jpg")  
  
# Image dimensions  
h, w = img.shape[:2]  
  
# Image center  
cx = w / 2  
cy = h / 2  
  
# 300% means 3 times the original size  
sx = 3.0  
sy = 3.0  
  
# The 2 x 2 scaling matrix requested by the task  
S = np.array([  
    [sx, 0],  
    [0, sy]  
, dtype=np.float32)  
  
print("2 x 2 Scaling Matrix:")  
print(S)  
  
# Translate so scaling happens around the image center  
tx = cx - sx * cx  
ty = cy - sy * cy  
  
# OpenCV warpAffine expects a 2 x 3 affine matrix.  
# The final column contains translation.  
M = np.array([  
    [sx, 0, tx],  
    [0, sy, ty]
```

```
], dtype=np.float32)

# Keep the same output canvas size
scaled = cv2.warpAffine(
    img,
    M,
    (w, h)
)

cv2.imshow("Original", img)
cv2.imshow("Scaled and Centered", scaled)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Function dictionary

cv2.imread(filename)

- `filename` : image path/name.
- returns the loaded image or `None` if loading fails.

np.array(values, dtype=...)

- `values` : nested Python list/matrix.
- `dtype` : desired NumPy datatype.

cv2.warpAffine(src, M, dsize)

- `src` : input image.
- `M` : 2 x 3 affine transformation matrix.
- `dsize` : output size as `(width, height)`.

cv2.imshow(window_name, image)

- `window_name` : title of window.
- `image` : image to display.

`cv2.waitKey(delay)`

- `delay` : milliseconds to wait; `0` = wait indefinitely.

`cv2.destroyAllWindows()`

- closes OpenCV display windows.
-

PART D - TASK 2: THE DIZZY SATELLITE

What the task asks

- The image was rotated by +45 degrees.
- Formulate a 2 x 2 rotation matrix.
- Rotate it back to true north.
- Calculate new dimensions manually so corners are not cropped.

Approach

Undo +45 degrees with:

```
theta = -45 degrees
```

Rotation matrix

```
R(-45) =  
[ cos(-45)  -sin(-45)  
  sin(-45)   cos(-45) ]  
  
=  
[ 0.7071   0.7071  
 -0.7071   0.7071 ]
```

New dimensions

For an image of width `w` and height `h` :

```
new_w = |w*cos(theta)| + |h*sin(theta)|  
new_h = |h*cos(theta)| + |w*sin(theta)|
```

Use ceiling when converting to integer dimensions.

Complete solution

```
import cv2  
import numpy as np  
  
img = cv2.imread("satellite.jpg")  
  
if img is None:  
    raise FileNotFoundError("Could not find satellite.jpg")  
  
h, w = img.shape[:2]  
  
# The image was rotated +45 degrees.  
# Rotate by -45 degrees to undo it.  
angle = -45  
theta = np.deg2rad(angle)  
  
# 2 x 2 rotation matrix  
R = np.array([  
    [np.cos(theta), -np.sin(theta)],  
    [np.sin(theta), np.cos(theta)]  
], dtype=np.float32)  
  
print("2 x 2 Rotation Matrix:")  
print(R)  
  
# Calculate the bounding-box dimensions after rotation.  
# This prevents the rotated corners from being cropped.  
new_w = int(np.ceil(  
    abs(w * np.cos(theta)) +  
    abs(h * np.sin(theta))  
))  
  
new_h = int(np.ceil(  
    abs(h * np.cos(theta)) +  
    abs(w * np.sin(theta))  
))
```

```
    abs(h * np.cos(theta)) +
    abs(w * np.sin(theta))
))

# Rotate about the image center
center = (w / 2, h / 2)

M = cv2.getRotationMatrix2D(
    center,
    angle,
    1.0
)

# Recenter the rotated image inside the new canvas
M[0, 2] += new_w / 2 - center[0]
M[1, 2] += new_h / 2 - center[1]

rotated = cv2.warpAffine(
    img,
    M,
    (new_w, new_h)
)

cv2.imshow("Original", img)
cv2.imshow("Corrected Satellite", rotated)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Function dictionary

np.deg2rad(x)

- `x` : angle in degrees.
- returns radians.

np.cos(x)

- `x` : angle in radians.
- returns cosine.

np.sin(x)

- `x` : angle in radians.
- returns sine.

np.ceil(x)

- `x` : number/array.
- rounds upward.

cv2.getRotationMatrix2D(center, angle, scale)

- `center` : (x, y) rotation center.
- `angle` : rotation angle in degrees.
- `scale` : scale factor, usually `1.0` for rotation only.
- returns a 2 x 3 affine rotation matrix.

abs(x)

- absolute value.

PART E - TASK 3: THE FAST TRAIN BARCODE

What the task asks

- Construct a 2 x 2 horizontal shear matrix.
- Push pixels horizontally.
- Undo the slant so the barcode becomes rectangular and readable.

The task does not provide a numerical shear coefficient `k`, so the code keeps `k` as an input.

Approach

Forward horizontal shear:

$$H = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$$

If the image was distorted using `+k`, undo it using:

$$H_{\text{inverse}} = \begin{bmatrix} 1 & -k \\ 0 & 1 \end{bmatrix}$$

Complete solution

```
import cv2
import numpy as np

img = cv2.imread("barcode.jpg")

if img is None:
    raise FileNotFoundError("Could not find barcode.jpg")

h, w = img.shape[:2]

# Enter the shear amount measured/given in the question.
k = float(input("Enter shear k: "))

# Inverse horizontal shear removes a rightward shear of +k.
S = np.array([
    [1, -k],
    [0, 1]
], dtype=np.float32)

print("2 x 2 Inverse Shear Matrix:")
print(S)

# Find where image corners move so we can make
# an output canvas large enough for the whole result.
corners = np.float32([
    [0, 0],
    [w, 0],
    [0, h],
```

```

        [w, h]
    ])

    transformed_corners = corners @ S.T

    min_x = transformed_corners[:, 0].min()
    min_y = transformed_corners[:, 1].min()
    max_x = transformed_corners[:, 0].max()
    max_y = transformed_corners[:, 1].max()

    # Shift the result if any coordinates become negative.
    tx = -min_x
    ty = -min_y

    # Convert the 2 x 2 shear matrix to OpenCV's 2 x 3 format.
    M = np.array([
        [1, -k, tx],
        [0, 1, ty]
    ], dtype=np.float32)

    new_w = int(np.ceil(max_x - min_x))
    new_h = int(np.ceil(max_y - min_y))

    corrected = cv2.warpAffine(
        img,
        M,
        (new_w, new_h)
    )

    cv2.imshow("Slanted Barcode", img)
    cv2.imshow("Corrected Barcode", corrected)

    cv2.waitKey(0)
    cv2.destroyAllWindows()

```

Function dictionary

`corners @ S.T`

- `@` : matrix multiplication.
- `S.T` : transpose of `S`.

.min()

- returns minimum element of an array.

.max()

- returns maximum element of an array.

cv2.warpAffine(src, M, dsize)

- `src` : image.
- `M` : 2 x 3 affine matrix.
- `dsize` : (width, height) .

PART F - TASK 4: THE HIDDEN TREASURE MAP

What the task asks

The map is misaligned. The X mark is hidden exactly:

```
150 pixels to the left
80 pixels up
```

The task asks you to upgrade from 2 x 2 to 3 x 3 homogeneous coordinates and translate the image back into view.

Because image y increases downward, moving the map back from "80 up" requires a positive `+80` downward shift.

Translation matrix

```
T =
[ 1   0  150
  0   1   80
  0   0   1 ]
```

Complete solution

```
import cv2
import numpy as np

img = cv2.imread("treasure_map.jpg")

if img is None:
    raise FileNotFoundError("Could not find treasure_map.jpg")

# Move the map right by 150 pixels
# and down by 80 pixels to bring the hidden mark back.
tx = 150
ty = 80

# 3 x 3 homogeneous translation matrix
T = np.array([
    [1, 0, tx],
    [0, 1, ty],
    [0, 0, 1]
], dtype=np.float32)

print("3 x 3 Translation Matrix:")
print(T)

# warpAffine needs only the first two rows.
M = T[:2, :]

h, w = img.shape[:2]

translated = cv2.warpAffine(
    img,
    M,
    (w, h)
)

cv2.imshow("Original Map", img)
cv2.imshow("Recovered Map", translated)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Function dictionary

`T[:2, :]`

- selects the first two rows and all columns.
- converts the 3 x 3 homogeneous matrix to the 2 x 3 matrix expected by `warpAffine`.

`cv2.warpAffine(src, M, dsize)`

- `src` : image.
- `M` : 2 x 3 affine matrix.
- `dsize` : (width, height) .

PART G - TASK 5: THE ROBOT'S ASSEMBLY LINE

What the task asks

A microchip is rotated sideways and slightly off-center.

The task asks you to:

- combine Rotation and Translation into one 3 x 3 matrix,
- apply the combined rigid transformation,
- align the chip with the robot's gripper.

The task sheet does not provide numeric angle/translation values, so use the values given in the question/exam.

Approach

Rotation:

```
R =
[ cos(theta)  -sin(theta)  0
  sin(theta)   cos(theta)  0
    0           0         1 ]
```

Translation:

```
T =
[ 1  0  tx
  0  1  ty
  0  0  1 ]
```

Rotate first, then translate:

```
H_rigid = T @ R
```

Resulting matrix form

```
H_rigid =
[ cos(theta)  -sin(theta)  tx
  sin(theta)   cos(theta)  ty
    0           0         1 ]
```

Complete solution

```
import cv2
import numpy as np

img = cv2.imread("microchip.jpg")

if img is None:
    raise FileNotFoundError("Could not find microchip.jpg")

# Values should come from the exam/task instance.
angle_deg = float(input("Enter rotation angle (degrees): "))
tx = float(input("Enter translation in x: "))
ty = float(input("Enter translation in y: "))

theta = np.deg2rad(angle_deg)
```

```

# Homogeneous rotation matrix
R = np.array([
    [np.cos(theta), -np.sin(theta), 0],
    [np.sin(theta),  np.cos(theta), 0],
    [0,              0,           1]
], dtype=np.float32)

# Homogeneous translation matrix
T = np.array([
    [1, 0, tx],
    [0, 1, ty],
    [0, 0, 1]
], dtype=np.float32)

# Right-to-left application:
# R happens first, then T.
H_rigid = T @ R

print("3 x 3 Rigid Transformation Matrix:")
print(H_rigid)

# warpAffine only needs the first two rows.
M = H_rigid[:2, :]

h, w = img.shape[:2]

aligned = cv2.warpAffine(
    img,
    M,
    (w, h)
)

cv2.imshow("Original Chip", img)
cv2.imshow("Aligned Chip", aligned)

cv2.waitKey(0)
cv2.destroyAllWindows()

```

Function dictionary

cv2.deg2rad / np.deg2rad

Use:

```
np.deg2rad(angle_deg)
```

- `angle_deg` : degree value.
- returns radians.

A @ B

- matrix multiplication.
- order matters.

```
cv2.warpAffine(src, M, dsize)
```

- `M` : 2 x 3 affine matrix.
- `dsize` : output (width, height) .

PART H - TASK 6: THE ARCHITECT'S BLUEPRINT

What the task asks

The blueprint is:

- too small,
- rotated randomly,
- in the wrong location.

The task asks you to combine:

```
Scale + Rotation + Translation
```

into one 3 x 3 operation.

Approach

Use uniform scaling:

```
S =
[ s  0  0
  0  s  0
  0  0  1 ]
```

Rotation:

```
R =
[ cos(theta)  -sin(theta)  0
  sin(theta)   cos(theta)  0
      0          0        1 ]
```

Translation:

```
T =
[ 1  0  tx
  0  1  ty
  0  0  1 ]
```

Sequence:

```
scale -> rotate -> translate
```

Therefore:

```
H_similarity = T @ R @ S
```

Complete solution

```
import cv2
import numpy as np

img = cv2.imread("blueprint.jpg")
```

```

if img is None:
    raise FileNotFoundError("Could not find blueprint.jpg")

# Values come from the actual question/exam.
scale = float(input("Enter uniform scale factor: "))
angle_deg = float(input("Enter rotation angle (degrees): "))
tx = float(input("Enter translation in x: "))
ty = float(input("Enter translation in y: "))

theta = np.deg2rad(angle_deg)

# Uniform scaling: same factor on x and y.
S = np.array([
    [scale, 0, 0],
    [0, scale, 0],
    [0, 0, 1]
], dtype=np.float32)

# Rotation around the origin.
R = np.array([
    [np.cos(theta), -np.sin(theta), 0],
    [np.sin(theta), np.cos(theta), 0],
    [0, 0, 1]
], dtype=np.float32)

# Translation.
T = np.array([
    [1, 0, tx],
    [0, 1, ty],
    [0, 0, 1]
], dtype=np.float32)

# Apply in this order:
# S first -> R second -> T last.
H = T @ R @ S

print("3 x 3 Similarity Matrix:")
print(H)

M = H[:2, :]

h, w = img.shape[:2]

result = cv2.warpAffine(
    img,
    M,

```

```
(w, h)
)

cv2.imshow("Original Blueprint", img)
cv2.imshow("Corrected Blueprint", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Function dictionary

`np.cos(theta)`

- input must be radians.

`np.sin(theta)`

- input must be radians.

`cv2.warpAffine(src, M, dsize)`

- `src` : image.
- `M` : 2 x 3 affine matrix.
- `dsize` : (width, height) .

PART I - TASK 7: THE GLITCHED IMAGE

What the task asks

Three landmarks have known:

```
starting coordinates
ending coordinates
```

The image was squished, slanted, rotated, and moved.

You must solve the six unknown parameters of a general affine matrix:

a, b, tx, c, d, ty

Core equations

$$\begin{aligned}x' &= a*x + b*y + tx \\ y' &= c*x + d*y + ty\end{aligned}$$

For each point:

$$\begin{bmatrix} x & y & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x & y & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} x' \\ y' \end{bmatrix}$$

Three point pairs give six equations.

Complete solution

```
import cv2
import numpy as np

img = cv2.imread("glitched.jpg")

if img is None:
    raise FileNotFoundError("Could not find glitched.jpg")

# Replace these with the 3 source landmarks
# and their 3 corresponding destination landmarks
# given by the exam question.
src = np.float32([
    [x1, y1],
    [x2, y2],
    [x3, y3]
])

dst = np.float32([
    [xp1, yp1],
    [xp2, yp2],
    [xp3, yp3]
])
```

```

A = []
B = []

# Each point pair contributes TWO equations:
# one for x' and one for y'.
for (x, y), (xp, yp) in zip(src, dst):

    # x' = a*x + b*y + tx
    A.append([x, y, 1, 0, 0, 0])
    B.append(xp)

    # y' = c*x + d*y + ty
    A.append([0, 0, 0, x, y, 1])
    B.append(yp)

A = np.array(A, dtype=np.float64)
B = np.array(B, dtype=np.float64)

# Solve A * parameters = B.
# Do NOT need to calculate the inverse manually.
params = np.linalg.solve(A, B)

a, b, tx, c, d, ty = params

print("Solved parameters:")
print("a =", a)
print("b =", b)
print("tx =", tx)
print("c =", c)
print("d =", d)
print("ty =", ty)

# Construct the resulting 3 x 3 affine matrix.
H = np.array([
    [a, b, tx],
    [c, d, ty],
    [0, 0, 1]
], dtype=np.float64)

print("Affine Matrix:")
print(H)

# OpenCV affine warp uses the top 2 rows.
M = H[:2, :].astype(np.float32)

```

```
h, w = img.shape[:2]

corrected = cv2.warpAffine(
    img,
    M,
    (w, h)
)

cv2.imshow("Glitched Image", img)
cv2.imshow("Corrected Image", corrected)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Important: `x1` , `y1` , etc. above are placeholders because the task sheet describes three correspondences but does not print numerical coordinates on the page.

Function dictionary

`zip(src, dst)`

- pairs corresponding source and destination points.

`np.linalg.solve(A, B)`

- solves:

$$A * x = B$$

- returns the unknown vector `x` .
- requires a valid square system for this exact usage.

`np.array(..., dtype=...)`

- creates NumPy arrays with the requested datatype.

`cv2.warpAffine(src, M, dsize)`

- applies the recovered affine mapping.

PART J - TASK 8: THE SIDEWALK ILLUSION

What the task asks

A 3D chalk artwork is photographed from an angle.

Because of perspective, the far side looks smaller than the near side.

The task asks you to:

- select four corners,
- calculate a perspective transformation,
- warp the image into a top-down bird's-eye square.

Approach

Use FOUR corresponding points.

```
source quadrilateral -> destination square
```

Use:

```
cv2.getPerspectiveTransform()
```

to calculate the 3 x 3 homography.

Then:

```
cv2.warpPerspective()
```

to apply it.

Complete solution

```
import cv2
import numpy as np

img = cv2.imread("chalk_art.jpg")

if img is None:
    raise FileNotFoundError("Could not find chalk_art.jpg")

# Four corners of the distorted artwork.
# Replace these values with the actual four corners.
# Keep the SAME order in source and destination.
src = np.float32([
    [x1, y1], # top-left
    [x2, y2], # top-right
    [x3, y3], # bottom-right
    [x4, y4]  # bottom-left
])

# Desired square output.
side = 500

dst = np.float32([
    [0, 0],
    [side - 1, 0],
    [side - 1, side - 1],
    [0, side - 1]
])

# Calculate the 3 x 3 perspective/homography matrix.
H = cv2.getPerspectiveTransform(src, dst)

print("3 x 3 Homography Matrix:")
print(H)

# Warp the quadrilateral into a square.
birds_eye = cv2.warpPerspective(
    img,
    H,
    (side, side)
)

cv2.imshow("Original Perspective", img)
cv2.imshow("Bird's Eye View", birds_eye)
```

```
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Function dictionary

`cv2.getPerspectiveTransform(src, dst)`

- `src` : four source points as `float32` .
- `dst` : four destination points as `float32` .
- returns a 3 x 3 perspective transformation matrix.

`cv2.warpPerspective(src, M, dsize)`

- `src` : input image.
- `M` : 3 x 3 homography matrix.
- `dsize` : (width, height) .

`np.float32(values)`

- converts values to 32-bit floating point, useful for OpenCV point arrays.

PART K - TASK 9: THE STADIUM PANORAMA

What the task asks

Two cameras took overlapping photos of the same football field.

You must:

1. choose 4 matching points in both images,
2. calculate the homography,
3. project camera 2 into camera 1's coordinate space,

4. stitch them into one panorama.

Approach

If:

```
pts2 = points in camera/image 2  
pts1 = corresponding points in camera/image 1
```

then compute:

```
H, mask = cv2.findHomography(pts2, pts1, 0)
```

This mapping sends:

```
image 2 -> image 1
```

Then warp image 2.

Complete solution

```
import cv2  
import numpy as np  
  
img1 = cv2.imread("stadium1.jpg")  
img2 = cv2.imread("stadium2.jpg")  
  
if img1 is None or img2 is None:  
    raise FileNotFoundError("Could not load one or both stadium images")  
  
# Four matching points.  
# pts2 and pts1 MUST correspond in the same order.  
# Replace the placeholders with the actual points.  
pts2 = np.float32([  
    [x1_2, y1_2],  
    [x2_2, y2_2],  
    [x3_2, y3_2],  
    [x4_2, y4_2]  
])
```

```

pts1 = np.float32([
    [x1_1, y1_1],
    [x2_1, y2_1],
    [x3_1, y3_1],
    [x4_1, y4_1]
])

# Find H that maps image 2 coordinates into image 1 coordinates.
H, mask = cv2.findHomography(
    pts2,
    pts1,
    0
)

print("Homography Matrix:")
print(H)

h1, w1 = img1.shape[:2]
h2, w2 = img2.shape[:2]

# Four corners of image 2
corners2 = np.float32([
    [0, 0],
    [w2, 0],
    [w2, h2],
    [0, h2]
]).reshape(-1, 1, 2)

# Where those corners land after H
warped_corners2 = cv2.perspectiveTransform(
    corners2,
    H
).reshape(-1, 2)

# Image 1 corners in its own coordinate system
corners1 = np.float32([
    [0, 0],
    [w1, 0],
    [w1, h1],
    [0, h1]
])

# Find complete panorama bounds.
all_corners = np.vstack([
    corners1,
    warped_corners2
])

```

```

])

min_x = int(np.floor(all_corners[:, 0].min()))
min_y = int(np.floor(all_corners[:, 1].min()))

max_x = int(np.ceil(all_corners[:, 0].max()))
max_y = int(np.ceil(all_corners[:, 1].max()))

panorama_w = max_x - min_x
panorama_h = max_y - min_y

# Translation moves negative coordinates into the visible canvas.
T = np.array([
    [1, 0, -min_x],
    [0, 1, -min_y],
    [0, 0, 1]
], dtype=np.float64)

# Apply H first, then translate into panorama canvas.
H_panorama = T @ H

# Warp image 2.
warped2 = cv2.warpPerspective(
    img2,
    H_panorama,
    (panorama_w, panorama_h)
)

# Create canvas and place image 1.
panorama = np.zeros_like(warped2)

x_offset = -min_x
y_offset = -min_y

panorama[
    y_offset:y_offset + h1,
    x_offset:x_offset + w1
] = img1

# Create masks for blending.
mask1 = np.zeros(
    (panorama_h, panorama_w),
    dtype=np.uint8
)

mask1[

```

```

        y_offset:y_offset + h1,
        x_offset:x_offset + w1
    ] = 255

mask2_original = np.ones(
    (h2, w2),
    dtype=np.uint8
) * 255

mask2 = cv2.warpPerspective(
    mask2_original,
    H_panorama,
    (panorama_w, panorama_h)
)

# Blend overlapping areas by averaging.
p1 = panorama.astype(np.float32)
p2 = warped2.astype(np.float32)

m1 = mask1 > 0
m2 = mask2 > 0

overlap = m1 & m2
p1[overlap] = (p1[overlap] + p2[overlap]) / 2

only2 = (~m1) & m2
p1[only2] = p2[only2]

panorama = np.clip(
    p1,
    0,
    255
).astype(np.uint8)

cv2.imshow("Stadium Panorama", panorama)

cv2.waitKey(0)
cv2.destroyAllWindows()

```

Function dictionary

cv2.findHomography(srcPoints, dstPoints, method)

- **srcPoints** : corresponding source points.

- `dstPoints` : corresponding destination points.
- `method` : method/robust estimator; `0` means a basic direct solve.
- returns `(H, mask)` .

`cv2.perspectiveTransform(src, M)`

- `src` : points to transform.
- `M` : 3 x 3 homography.
- returns transformed points.

`np.vstack(arrays)`

- vertically stacks arrays.

`np.floor(x) / np.ceil(x)`

- round down/up.

`np.zeros(shape, dtype)`

- creates an array filled with zeros.

`np.ones(shape, dtype)`

- creates an array filled with ones.

`np.clip(x, min, max)`

- limits values to a fixed range.

Important direction rule

```
findHomography(pts2, pts1)
```

means:

```
image 2 -> image 1
```

Switching the arguments reverses the mapping.

PART L - TASK 10: THE MASTER FORGER

What the task asks

Use the entire hierarchy:

1. **Linear Scale** to shrink the painting.
2. **Rigid Transformation** to rotate and move it near the frame.
3. **Projective Transformation** to map the painting's four corners into the perspective of the angled frame.

Main mathematical idea

Scale:

```
S
```

Rigid:

```
T @ R
```

Affine/similarity stage:

```
H_affine = T @ R @ S
```

Then use the transformed four painting corners and the four frame corners to calculate projective correction:

```
P = getPerspectiveTransform(intermediate_corners, frame_corners)
```

The complete transformation is:

$$H_{\text{total}} = P @ H_{\text{affine}}$$

Read the order right-to-left:

S -> R -> T -> P

Complete solution

```
import cv2
import numpy as np

painting = cv2.imread("painting.jpg")
frame = cv2.imread("frame.jpg")

if painting is None:
    raise FileNotFoundError("Could not find painting.jpg")

if frame is None:
    raise FileNotFoundError("Could not find frame.jpg")

# These values must come from the actual question/exam instance.
scale = float(input("Enter scale factor: "))
angle_deg = float(input("Enter rotation angle (degrees): "))
tx = float(input("Enter translation in x: "))
ty = float(input("Enter translation in y: "))

theta = np.deg2rad(angle_deg)

# -----
# 1. LINEAR SCALE
# -----
S = np.array([
    [scale, 0, 0],
    [0, scale, 0],
    [0, 0, 1]
], dtype=np.float64)

# -----
# 2. ROTATION
# -----
```

```

R = np.array([
    [np.cos(theta), -np.sin(theta), 0],
    [np.sin(theta),  np.cos(theta), 0],
    [0,          0,          1]
], dtype=np.float64)

# -----
# 3. TRANSLATION
# -----
T = np.array([
    [1, 0, tx],
    [0, 1, ty],
    [0, 0, 1]
], dtype=np.float64)

# Scale -> rotate -> translate
H_affine = T @ R @ S

# Four corners of the original painting
hp, wp = painting.shape[:2]

painting_corners = np.float32([
    [0, 0],
    [wp - 1, 0],
    [wp - 1, hp - 1],
    [0, hp - 1]
]).reshape(-1, 2)

# Find where the corners land after scale + rotation + translation.
intermediate_corners = cv2.perspectiveTransform(
    painting_corners,
    H_affine
).reshape(-1, 2)

# -----
# 4. FOUR CORNERS OF THE ANGLED FRAME
#
# Replace the placeholders with the actual frame corners.
# Keep the same corner order as the painting corners.
# -----
frame_corners = np.float32([
    [fx1, fy1], # top-left
    [fx2, fy2], # top-right
    [fx3, fy3], # bottom-right
    [fx4, fy4]  # bottom-left
])

```

```

# Perspective matrix maps the intermediate painting corners
# exactly onto the frame corners.
P = cv2.getPerspectiveTransform(
    intermediate_corners.astype(np.float32),
    frame_corners
)

print("Projective Matrix:")
print(P)

# -----
# 5. COMPLETE TRANSFORMATION
#
# Original painting:
# Scale -> Rotate -> Translate -> Perspective
# -----
H_total = P @ H_affine

print("Complete 3 x 3 Matrix:")
print(H_total)

hf, wf = frame.shape[:2]

# Apply complete projective transformation in one warp.
warped_painting = cv2.warpPerspective(
    painting,
    H_total,
    (wf, hf)
)

# Create a mask to know where the painting exists.
mask = np.ones(
    (hp, wp),
    dtype=np.uint8
) * 255

warped_mask = cv2.warpPerspective(
    mask,
    H_total,
    (wf, hf)
)

# Composite warped painting onto the frame.
result = frame.copy()

```

```
result[warped_mask > 0] = warped_painting[warped_mask > 0]

cv2.imshow("Original Frame", frame)
cv2.imshow("Painting in Frame", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Function dictionary

cv2.perspectiveTransform(points, H)

- `points` : input coordinates.
- `H` : 3 x 3 projective matrix.
- returns transformed coordinates.

cv2.getPerspectiveTransform(src, dst)

- exactly four source points.
- exactly four destination points.
- returns 3 x 3 homography.

cv2.warpPerspective(src, M, dsize)

- `src` : image.
- `M` : 3 x 3 projective matrix.
- `dsize` : output (width, height) .

result[mask > 0] = warped[mask > 0]

- replaces only the pixels where the mask says the transformed painting exists.

PART M - MASTER OPENCV / NUMPY

FUNCTION REFERENCE

Image input

```
cv2.imread(filename)
```

or:

```
cv2.imread(filename, flags)
```

Common flag:

```
cv2.IMREAD_GRAYSCALE
```

Affine warp

```
cv2.warpAffine(src, M, dsize)
```

- `src` = image.
- `M` = 2 x 3 affine matrix.
- `dsize` = (width, height).

Use for:

```
scale  
rotation  
translation  
shear  
general affine  
rigid  
similarity
```

Rotation matrix helper

```
cv2.getRotationMatrix2D(center, angle, scale)
```

- `center` = `(x, y)` rotation center.
 - `angle` = degrees.
 - `scale` = multiplicative scale.
 - returns 2 x 3 matrix.
-

Perspective matrix from four corners

```
cv2.getPerspectiveTransform(src, dst)
```

- `src` = four source points.
 - `dst` = four destination points.
 - returns 3 x 3 matrix.
-

Perspective warp

```
cv2.warpPerspective(src, M, dsize)
```

- `src` = image.
 - `M` = 3 x 3 homography.
 - `dsize` = `(width, height)`.
-

Homography estimation

```
cv2.findHomography(srcPoints, dstPoints, method)
```

- `srcPoints` = source correspondences.
- `dstPoints` = destination correspondences.
- `method` = estimation method; `0` is direct/basic solve.

Returns:

```
H, mask
```

Point transformation

```
cv2.perspectiveTransform(points, H)
```

- `points` = coordinates.
- `H` = 3 x 3 homography.
- returns transformed points.

NumPy matrix creation

```
np.array(values, dtype=np.float32)
```

- `values` = list/matrix.
- `dtype` = datatype.

Common:

```
np.float32  
np.float64  
np.uint8
```

Matrix multiplication

```
A @ B
```

Important:

```
A @ B != B @ A
```

in general.

For:

```
T @ R @ S
```

operations occur:

```
S -> R -> T
```

Linear-system solving

```
np.linalg.solve(A, B)
```

Solves:

```
A x = B
```

Useful in Task 7.

Reshaping

```
array.reshape(...)
```

Examples:

```
.reshape(-1, 1, 2)
```

for OpenCV point structures.

Rounding

```
np.ceil(x)  
np.floor(x)
```

Useful for calculating output canvas dimensions.

Array stacking

```
np.vstack((A, B))
```

Stacks arrays vertically.

Array creation

```
np.zeros(shape, dtype=...)  
np.ones(shape, dtype=...)  
np.zeros_like(array)
```

PART N - WHAT EACH TRANSFORMATION NEEDS

Question wording	Think immediately
Enlarge / shrink	Scale
Turn / rotate	Rotation
Slant / skew	Shear
Move / shift	Translation
Rotate + move, shape/size unchanged	Rigid
Scale + rotate + move, uniform scaling	Similarity
Three point pairs + six unknowns	General Affine
Four corners -> square/top-down	Perspective / Homography
Two overlapping images	Homography
Scale -> rigid -> perspective	Full hierarchy

PART O - WHICH OPENCV MATRIX SIZE?

2 x 2

-> pure linear mathematical form
scale / rotation / shear / reflection

2 x 3

-> OpenCV warpAffine matrix
affine + translation in image warping

3 x 3

-> homogeneous 2D affine matrix
translation can be represented with matrix multiplication

3 x 3

-> projective / homography matrix
perspective

4 x 4
-> homogeneous 3D transformations

Very common MCQ:

cv2.warpAffine -> 2 x 3
cv2.warpPerspective -> 3 x 3

PART P - THE MOST IMPORTANT MATRIX PATTERNS

Scaling

```
S = np.array([
    [sx, 0],
    [0, sy]
], dtype=np.float32)
```

Rotation

```
R = np.array([
    [np.cos(theta), -np.sin(theta)],
    [np.sin(theta), np.cos(theta)]
], dtype=np.float32)
```

Translation

```
T = np.array([
    [1, 0, tx],
    [0, 1, ty],
    [0, 0, 1]
], dtype=np.float32)
```

Horizontal shear

```
H = np.array([
    [1, k],
    [0, 1]
], dtype=np.float32)
```

Affine homogeneous

```
H = np.array([
    [a, b, tx],
    [c, d, ty],
    [0, 0, 1]
], dtype=np.float32)
```

Rigid

```
H = T @ R
```

Similarity

```
H = T @ R @ S
```

Full Task 10 hierarchy

```
H_total = P @ T @ R @ S
```

PART Q - POINT CORRESPONDENCE RULE

For Tasks 7, 8, 9, and 10:

Corresponding points must stay in the same order.

For example:

```
source:
1 = top-left
2 = top-right
3 = bottom-right
4 = bottom-left

destination:
1 = top-left
2 = top-right
3 = bottom-right
4 = bottom-left
```

Do NOT reorder one set independently.

PART R - MOST COMMON EXAM TRAPS

Trap 1: `(x, y)` vs `image[y, x]`

Mathematical point:

`(x, y)`

NumPy/OpenCV indexing:

`image[y, x]`

Trap 2: Forgetting that image y increases downward

```
right = +x
down  = +y
left  = -x
up    = -y
```

Trap 3: Forgetting radians for sin/cos

```
theta = np.deg2rad(angle_deg)
```

before:

```
np.sin(theta)  
np.cos(theta)
```

Trap 4: Wrong matrix order

If the required sequence is:

```
Scale -> Rotate -> Translate
```

write:

```
H = T @ R @ S
```

not:

```
H = S @ R @ T
```

Trap 5: Using a 2 x 2 matrix when translation is required

Pure linear:

```
p' = A p
```

cannot translate.

Use homogeneous 3 x 3:

$$p' = A p + b$$

or its 3 x 3 equivalent.

Trap 6: Using warpAffine for perspective

```
warpAffine      -> 2 x 3  
warpPerspective -> 3 x 3
```

Trap 7: Wrong homography direction

```
findHomography(pts2, pts1)
```

maps:

```
2 -> 1
```

Reversing the arguments maps:

```
1 -> 2
```

Trap 8: Cropping after rotation

If the question asks you to calculate new dimensions, use:

```
new_w = |w cos(theta)| + |h sin(theta)|  
new_h = |h cos(theta)| + |w sin(theta)|
```

PART S - 30-SECOND CODING CHECKLIST

1. What transformation is being requested?
2. Does it need 2 x 2, 2 x 3, or 3 x 3?
3. Is the operation linear, affine, rigid, similarity, or projective?
4. Do I need to undo the transformation?
5. If combining transformations, what is the required order?
6. Are the angles in degrees or radians?
7. Are points ordered consistently?
8. Does the output canvas need a new width/height?
9. Am I using warpAffine or warpPerspective?
10. What image/matrix/points are my INPUT and OUTPUT?

PART T - FAST MCQ MEMORY

Geometric transformation -> changes spatial coordinates

Image origin -> top-left

Image x -> right

Image y -> down

image[y, x] -> NumPy/OpenCV indexing

Linear -> $p' = A p$

Affine -> $p' = A p + b$

Linear origin rule -> $A0 = 0$

Translation -> needs translation vector / homogeneous coordinates

Scale matrix -> diagonal sx, sy

Rotation -> cos/sin matrix

Horizontal shear -> $\begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$

Affine preserves -> collinearity + parallelism

Projective -> perspective

Projective matrix -> 3×3

3D homogeneous matrix -> 4×4

warpAffine -> 2×3 matrix

warpPerspective -> 3×3 matrix

getPerspectiveTransform -> four point pairs -> 3×3 homography

findHomography -> corresponding point sets -> homography

$R @ S$ -> rotate result of scaling

$T @ R @ S$ -> scale, then rotate, then translate

Homography normalization -> divide x' and y' by w'

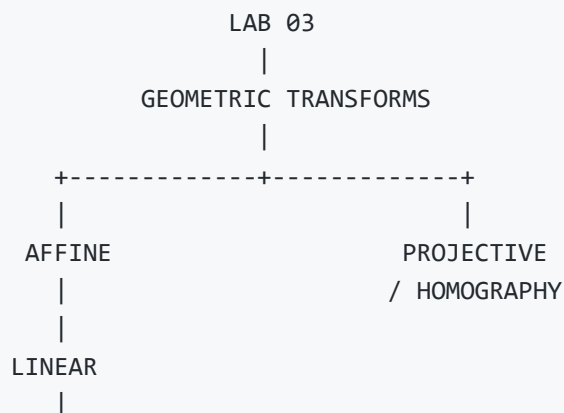
Task 7 -> 3 point pairs -> 6 affine unknowns

Task 8 -> four corners -> bird's-eye view

Task 9 -> two images -> homography -> panorama

Task 10 -> Scale -> Rigid -> Projective

PART U - FINAL LAB 03 MEMORY MAP





MUST be able to reproduce

63/65

```

    [np.sin(theta), np.cos(theta)]
], dtype=np.float32)

T = np.array([
    [1, 0, tx],
    [0, 1, ty],
    [0, 0, 1]
], dtype=np.float32)

H_rigid = T @ R

H_similarity = T @ R @ S

H = np.array([
    [a, b, tx],
    [c, d, ty],
    [0, 0, 1]
], dtype=np.float32)

```

and:

```

cv2.getPerspectiveTransform(src, dst)
cv2.warpPerspective(img, H, (width, height))

```

and:

```

H, mask = cv2.findHomography(src_points, dst_points, 0)

```

and:

```

params = np.linalg.solve(A, B)

```

MUST understand

- why homogeneous coordinates are needed
- why $A_0 = 0$ blocks translation
- matrix multiplication order
- image coordinate direction
- difference between affine and projective

- why rotation can crop corners
- why four corresponding points are used for perspective correction
- why matching point order matters

Nice to recognize, but not worth blind memorization

- every NumPy helper argument
- every display line
- every print statement
- exact variable names

The goal in an open-book coding exam is to recognize the transformation pattern quickly, then reconstruct the few lines that change from one question to another.